Attendance Code: 060013



UNIVERSITY OF LINCOLN

CMP2028M

Natural Language Processing

Week 9: “Overview of Neural Networks”

Heriberto Cuayahuitl

https://staff.Lincoln.ac.uk/hcuayahuitl

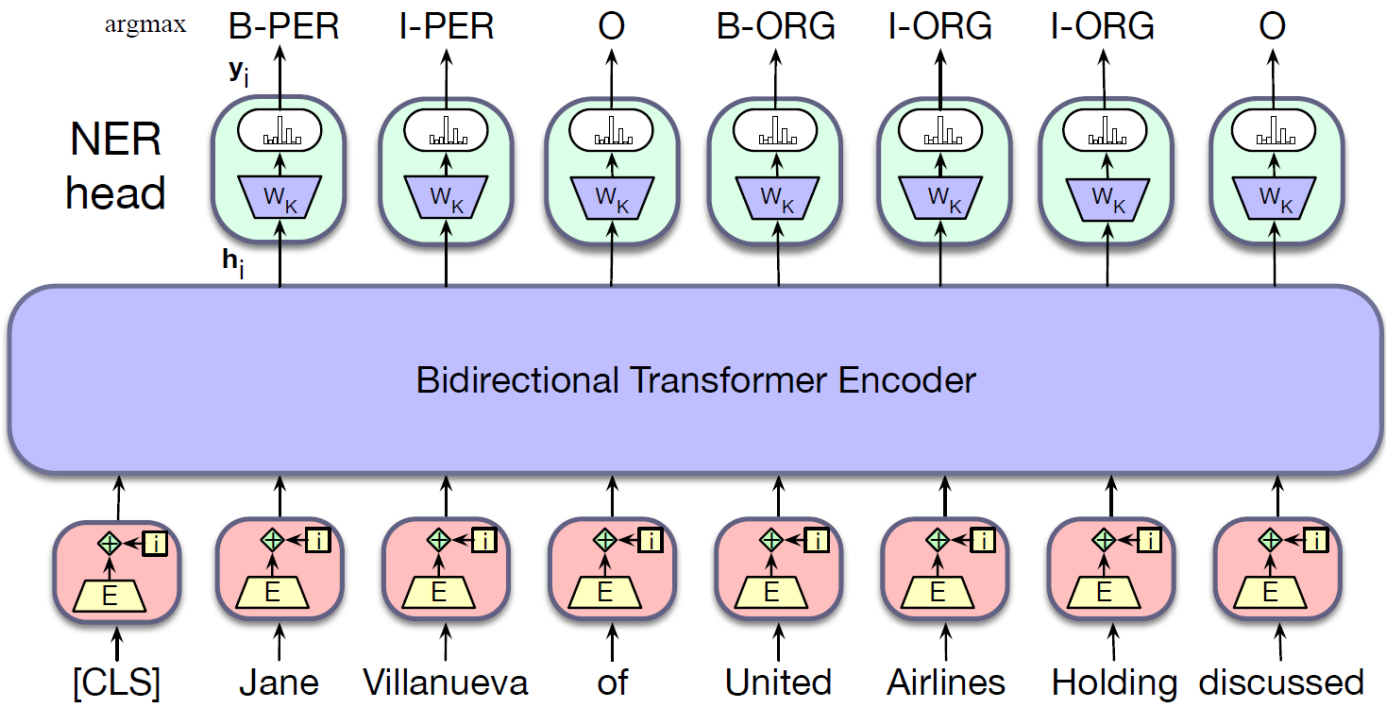
Agenda

- Week 1, 5-6 Feb: Tokenization and regular expressions
- Week 2, 12-13 Feb: N-gram language models
- Week 3, 19-20 Feb: Text classification
- Week 4, 27-27 Feb: Vector semantics and embeddings
- Week 5, 5-6 Mar: Large Language Models (LLMs)
- Week 6, 12-13 Mar: Question answering & information retrieval
- Week 7, 19-20 Mar: Constituency and dependency parsing
- Week 8, 26-27 Mar: Information extraction
- Week 9, 23-24 Apr: **Overview of neural networks**
- Week 10, 30 Apr-1May: Machine translation
- Week 11, 7-8 May: Chatbots and dialogue systems
- Week 12, 14-15 May: Speech recognition and synthesis

Easter break

Last Time (Before Easter)

- **Named Entity Recognition**



- Three Tasks:
  - Relation Extraction
  - Event Extraction
  - Time Extraction

Image from https://web.stanford.edu/~jurafsky/slp3/9.pdf

Today

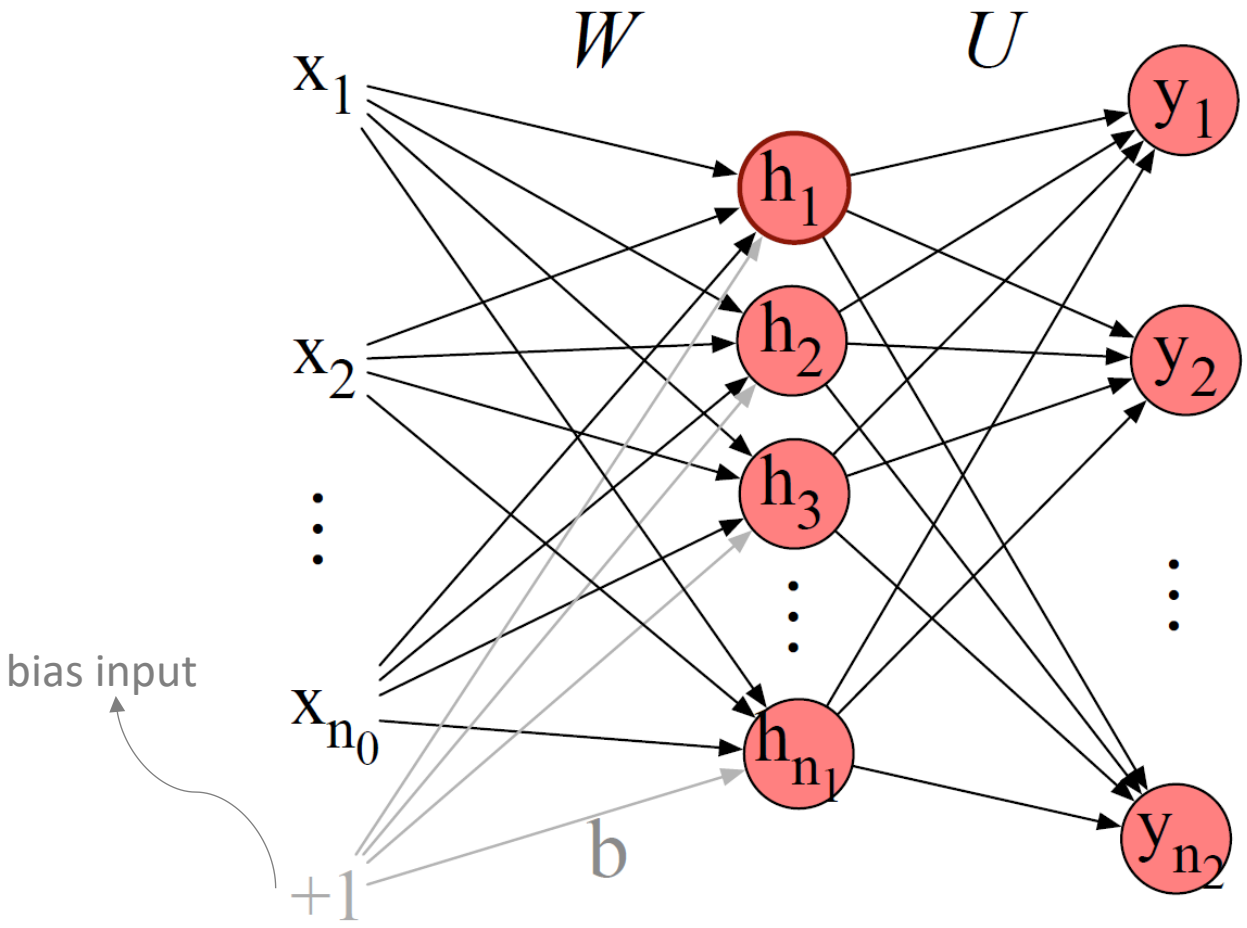
- **First step of the Backpropagation algorithm (out of two steps)**

- Transformer Networks
  - Main concepts
  - Focus on inference, not on training/finetuning

Acknowledgement: Some images from https://web.stanford.edu/~jurafsky/slp3/9.pdf

Multi-Layer Perceptron (MLP)

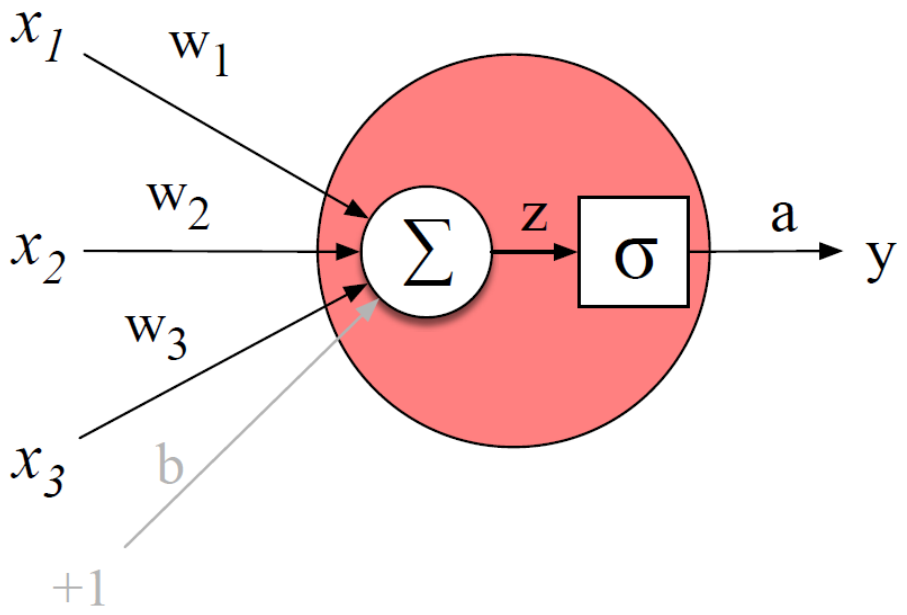
An MLP is a multi-layer neural network illustrated as:



An MLP is a type of feedforward neural network (FNN).

An Artificial Neuron

A neuron (computational unit) performs a weighted sum of its inputs, which is passed through a non-linear function.



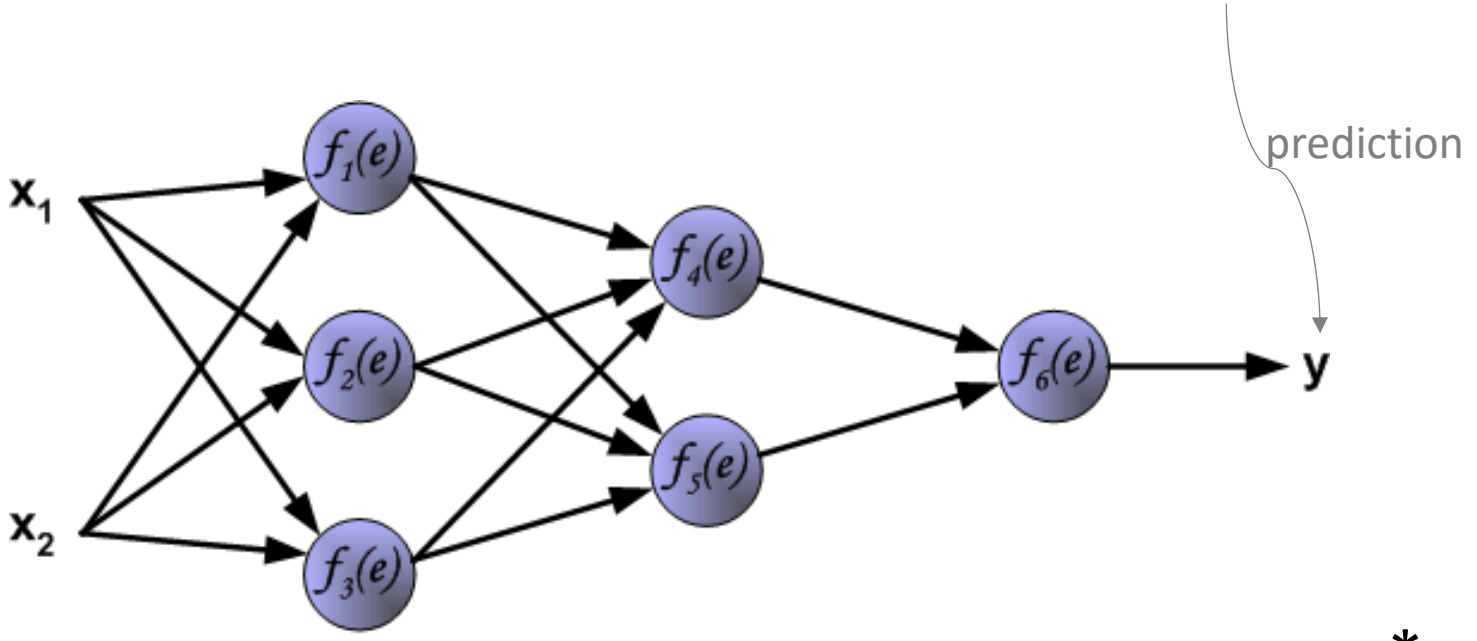
bias weight

Represented as $y = f(\sum_i w_i x_i + b) = f(\boldsymbol{w} \cdot \boldsymbol{x} + b)$

The bias weight is a learnt built-in offset, which can activate the neuron even if its

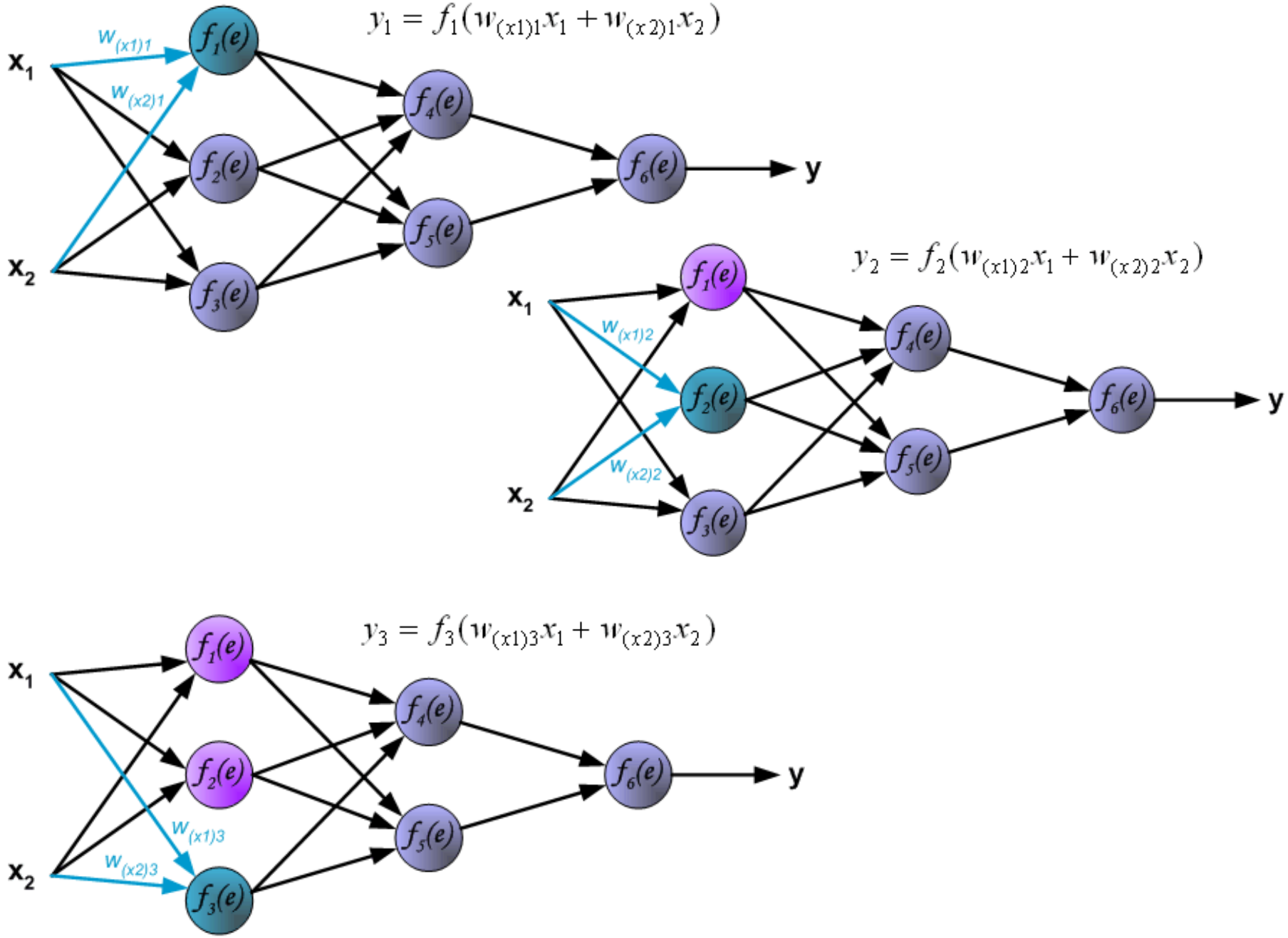
Forward Propagation

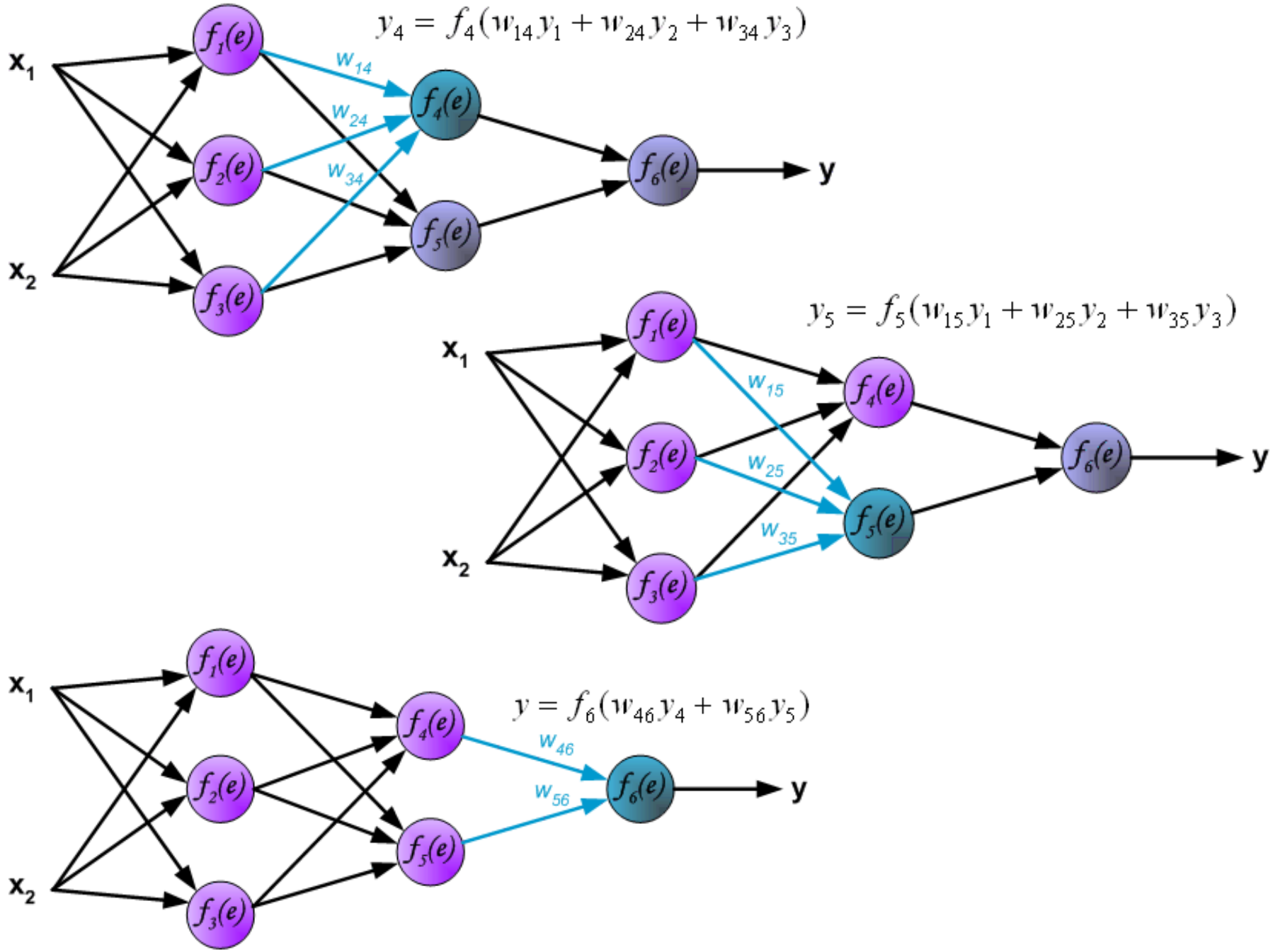
Forward propagation (*the forward step*) is the process of passing inputs through an MLP to produce an output.



f_i is an activation function to deal with non-linearities*.

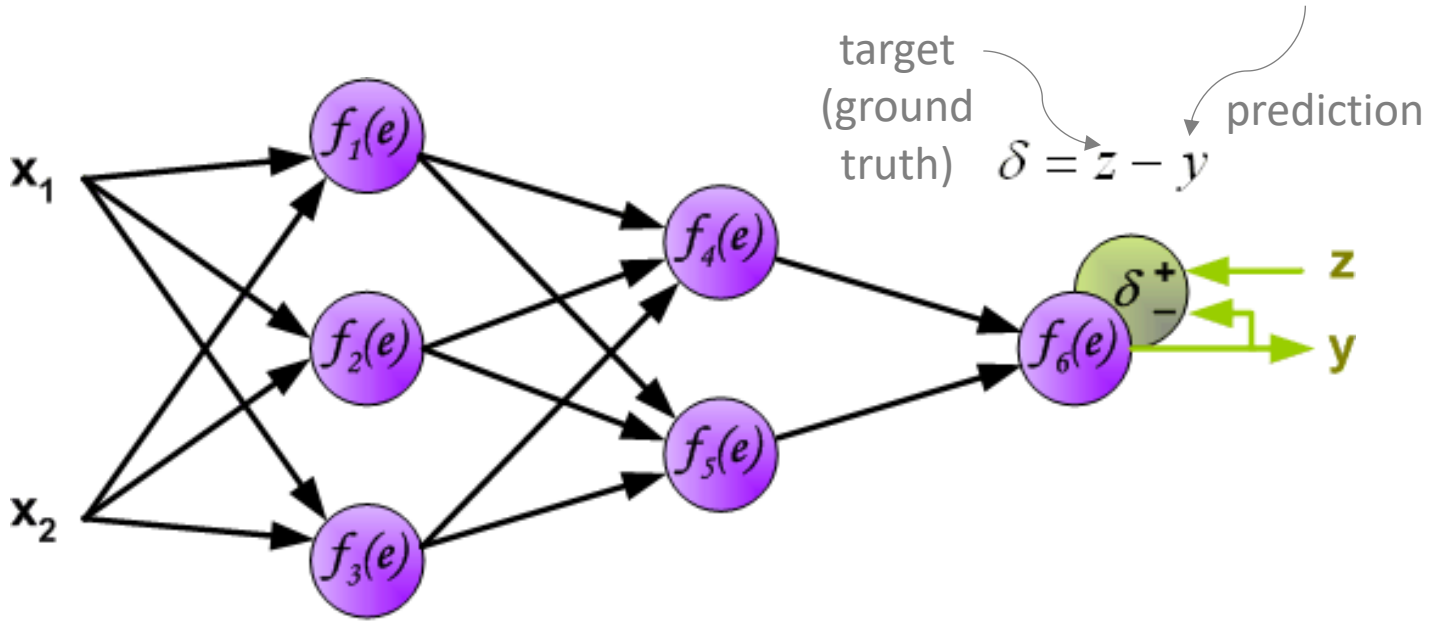
* They allow a neural net to model non-linear relationships between inputs and outputs


$y_1 = f_1(w_{(x1)1}x_1 + w_{(x2)1}x_2)$
$y_2 = f_2(w_{(x1)2}x_1 + w_{(x2)2}x_2)$
$y_3 = f_3(w_{(x1)3}x_1 + w_{(x2)3}x_2)$
x1
x2
y
$w_{(x1)1}$
$w_{(x2)1}$
$w_{(x1)2}$
$w_{(x2)2}$
$w_{(x1)3}$
$w_{(x2)3}$
$f_1(e)$
$f_2(e)$
$f_3(e)$
$f_4(e)$
$f_5(e)$
$f_6(e)$


$y_4 = f_4(w_{14}y_1 + w_{24}y_2 + w_{34}y_3)$
x1
x2
f1(e)
f2(e)
f3(e)
w14
w24
w34
f4(e)
f5(e)
f6(e)
y
$y_5 = f_5(w_{15}y_1 + w_{25}y_2 + w_{35}y_3)$
x1
x2
f1(e)
f2(e)
f3(e)
w15
w25
w35
f4(e)
f5(e)
f6(e)
y
$y = f_6(w_{46}y_4 + w_{56}y_5)$
x1
x2
f1(e)
f2(e)
f3(e)
f4(e)
f5(e)
w46
w56
f6(e)
y

Forward Propagation

Forward propagation (*the forward step*) is the process of passing inputs through an MLP to produce an output.

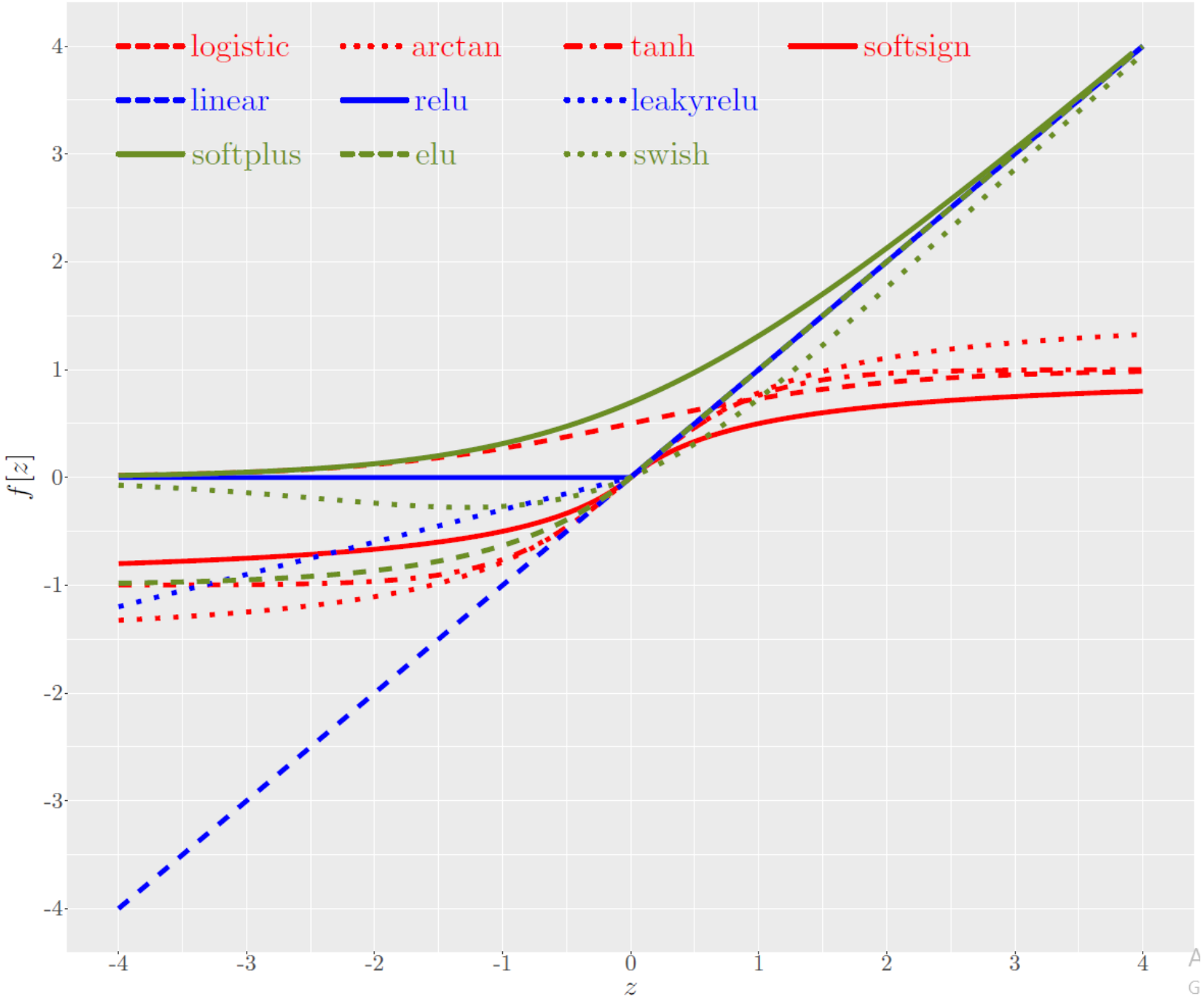


An MLP can have a single output for regression tasks or multiple outputs for classification tasks.

Continuous outpus

Discrete outpus

Activation Functions

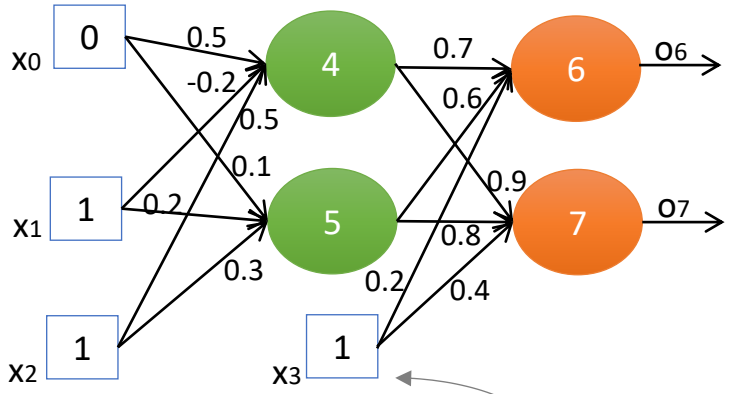


J. Lederer. "Activation Functions in Artificial Neural Networks:A Systematic

Example MLP: As a Classifier

Network:

$$o = \sigma\left(\sum_{j=1}^{n+1} w_j x_j\right) = \sigma(\boldsymbol{w}^T \boldsymbol{x})$$



x_2, x_3 = bias inputs

Hidden layer:

$$net_4 = 0.5 * 0 + (-0.2 * 1) + 0.5 * 1 = 0.3$$
$$net_5 = 0.1 * 0 + (0.2 * 1) + 0.3 * 1 = 0.5$$

$$o_4 = f(net_4) = Sigmoid(0.3) = 0.574$$
$$o_5 = f(net_5) = Sigmoid(0.5) = 0.622$$

$$\sigma(net) = \frac{1}{1 + e^{-net}}$$

Output layer:

$$net_6 = 0.7 * 0.574 + 0.6 * 0.622 + 0.2 * 1 = 0.975$$
$$net_7 = 0.9 * 0.574 + 0.8 * 0.622 + 0.4 * 1 = 1.414$$

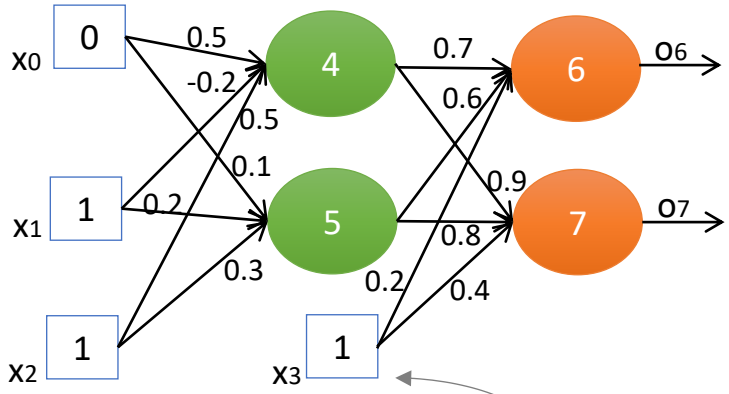
$$o_6 = \sigma(net_6) = 0.7261$$
$$o_7 = \sigma(net_7) = 0.8044$$

$$o_6 = Softmax(o_6) = 0.4804$$
$$o_7 = Softmax(o_7) = 0.5196$$

Example MLP: As a Regressor

Network:

$$o = \sigma\left(\sum_{j=1}^{n+1} w_j x_j\right) = \sigma(\boldsymbol{w}^T \boldsymbol{x})$$



x_2, x_3 = bias inputs

Hidden layer:

$$net_4 = 0.5 * 0 + (-0.2 * 1) + 0.5 * 1 = 0.3$$
$$net_5 = 0.1 * 0 + (0.2 * 1) + 0.3 * 1 = 0.5$$

$$o_4 = f(net_4) = Sigmoid(0.3) = 0.574$$
$$o_5 = f(net_5) = Sigmoid(0.5) = 0.622$$

$$\sigma(net) = \frac{1}{1 + e^{-net}}$$

Output layer:

$$net_6 = 0.7 * 0.574 + 0.6 * 0.622 + 0.2 * 1 = 0.975$$
$$net_7 = 0.9 * 0.574 + 0.8 * 0.622 + 0.4 * 1 = 1.414$$

~~$o_6 = \sigma(net_6) = 0.7261$~~
~~$o_7 = \sigma(net_7) = 0.8044$~~

$$o_6 = Softmax(o_6) = 0.975$$
$$o_7 = Softmax(o_7) = 1.414$$

Forward Propagation: Algorithm

Require: Network depth, l
Require: $\boldsymbol{W}^{(i)}, i \in \{1, \ldots, l\}$, the weight matrices of the model
Require: $\boldsymbol{b}^{(i)}, i \in \{1, \ldots, l\}$, the bias parameters of the model
Require: $\boldsymbol{x}$, the input to process
Require: $\boldsymbol{y}$, the target output

$\boldsymbol{h}^{(0)} = \boldsymbol{x}$
for $k = 1, \ldots, l$ **do**
  $\boldsymbol{a}^{(k)} = \boldsymbol{b}^{(k)} + \boldsymbol{W}^{(k)} \boldsymbol{h}^{(k-1)}$
  $\boldsymbol{h}^{(k)} = f(\boldsymbol{a}^{(k)})$
end for
$\hat{\boldsymbol{y}} = \boldsymbol{h}^{(l)}$

Algorithm from: Goodfellow et al. Deep Learning. 2016. Algorithm 6.3.

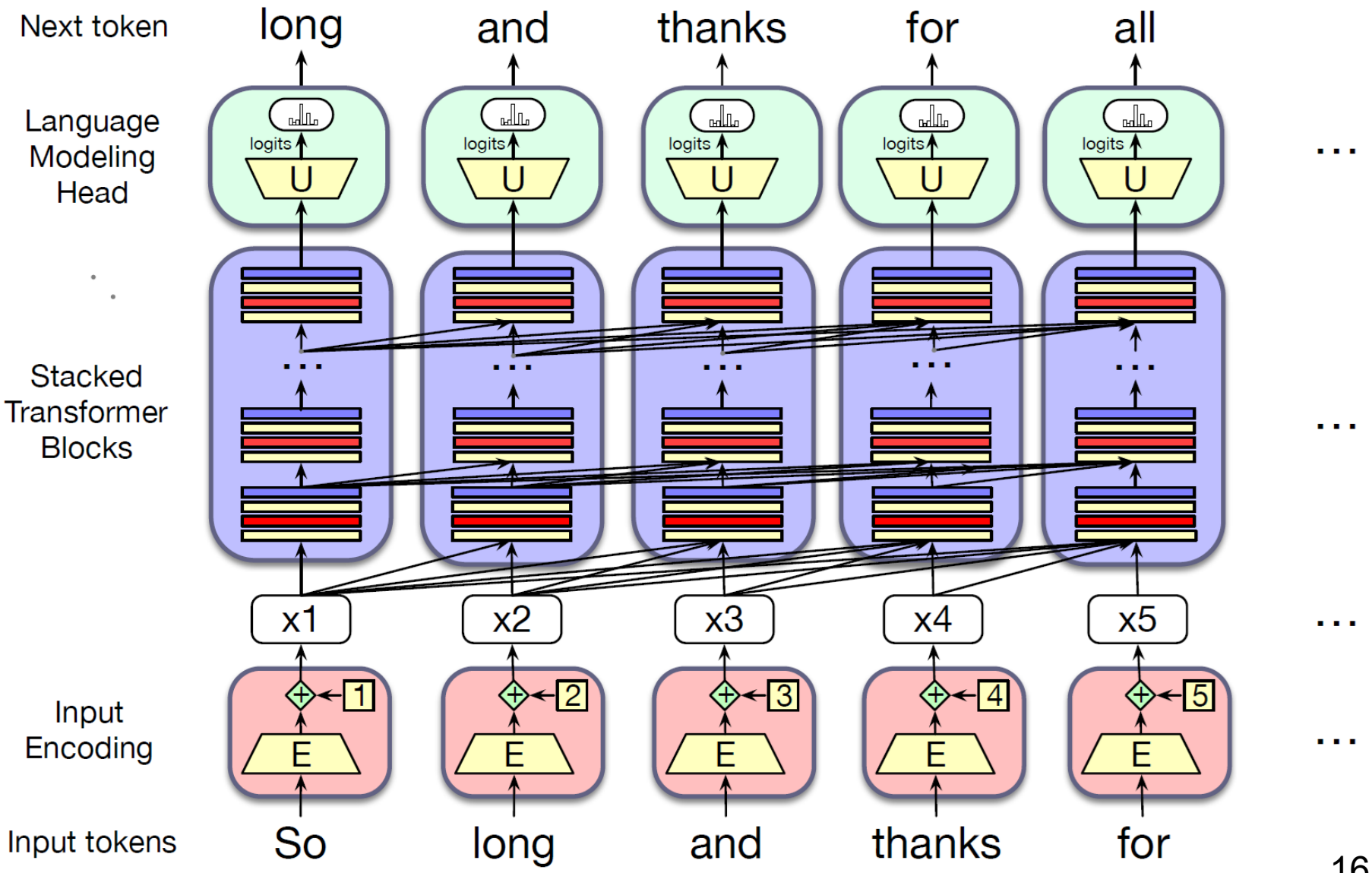
Today

- First step of the Backpropagation algorithm (out of two steps)

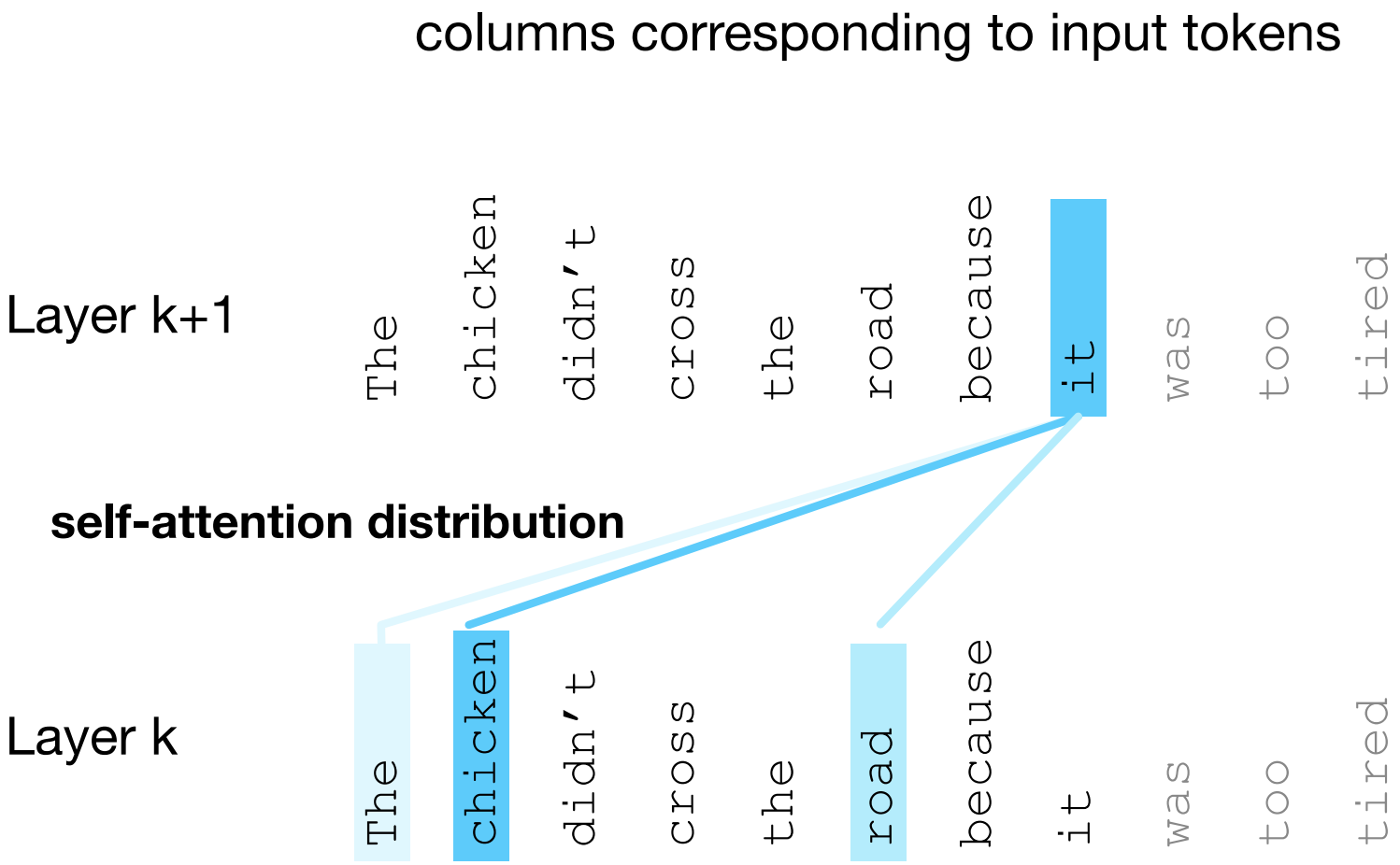
- **Transformer Networks**
  - Main concepts
  - Focus on inference, not on training/finetuning

Acknowledgement: Some images from https://web.stanford.edu/~jurafsky/slp3/9.pdf

Today: Transformer Networks

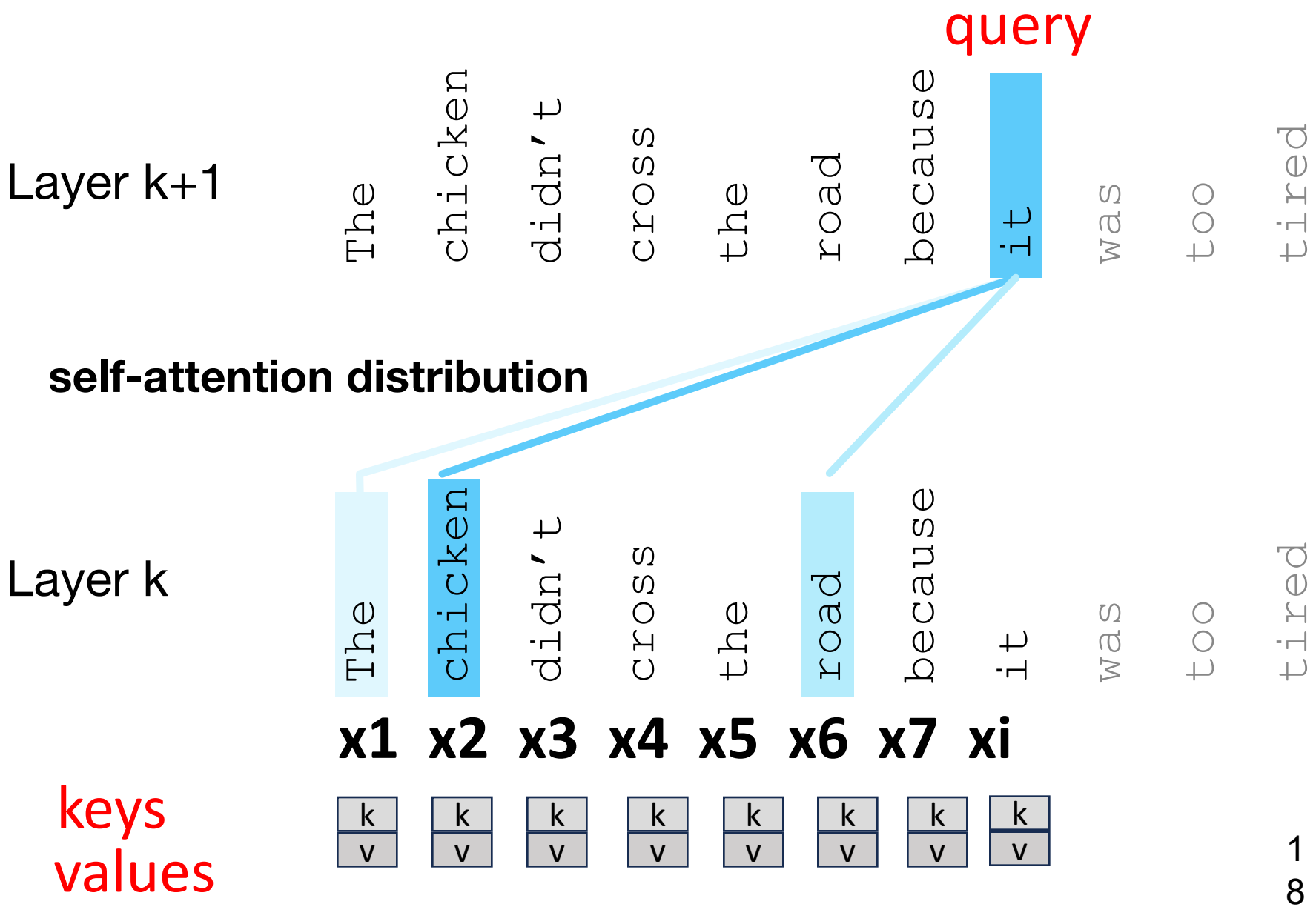


Intuition of Self-Attention



A word "**attends to**" some neighbouring words more than others

Intuition of Self-Attention



Self-Attention: Main Ideas

- **Query**: The representation of the current token (word) being compared to others. *Who should I pay attention to?*
- **Keys**: The representations of all words in the sequence that the query is comparing itself to for computing attention (similarity) scores.
- **Values**: Once the attention scores for all words are determined, a weighted sum of their values is output—giving more importance to words with higher attention.

Self-Attention: Calculation

We will use the following matrices to project each vector $\boldsymbol{x}_i$ into a representation of its role as query, key, value:

- **query:** $\boldsymbol{W^Q}$
- **key:** $\boldsymbol{W^K}$
- **value:** $\boldsymbol{W^V}$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}}; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{K}}; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{V}}$$

Self-Attention: Calculation

To compute similarity of current token x_i with some prior token x_j, we will use the dot product between q_i and k_j.

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}}; \quad \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^{\mathbf{K}}; \quad \mathbf{v}_j = \mathbf{x}_j \mathbf{W}^{\mathbf{V}}$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

Example of Self-Attention

```
import torch

x = torch.tensor([[1, 0, 0, 1, 2], [1, 0, 0, 1, 2], [0, 1, 1, 0, 0]], dtype=torch.float32)

WQ = torch.tensor([[2, 0, 0], [3, 0, 0], [1, 1, 0], [0, 1, 1], [1, 1, 1]], dtype=torch.float32)
WK = torch.tensor([[1, 2, 3], [2, 0, 0], [1, 2, 0], [0, 0, 3], [1, 2, 0]], dtype=torch.float32)
WV = torch.tensor([[1, 1, 1], [3, 0, 0], [1, 1, 0], [0, 1, 1], [2, 0, 0]], dtype=torch.float32)


q = torch.matmul(x, WQ)
k = torch.matmul(x, WK)
v = torch.matmul(x, WV)
print("q="+str(q))
print("k="+str(k))
print("v="+str(v))
unnormalised = torch.matmul(q, k.T) / k.shape[1]
probabilities = torch.nn.functional.softmax(unnormalised, dim=-1)
z = torch.matmul(probabilities, v)
print("unnormalised="+str(unnormalised))
print("probabilities="+str(probabilities))
print(z)
```


In the programs's output, note that equal attention is given to the first two input vectors.

Multi-Head Self-Attention

We apply the process above multiple times in parallel, with each head attending to different aspects of the context—to capture diverse relationships within the input.

$$\mathbf{q}_i^c = \mathbf{x}_i\mathbf{W}^{\mathbf{Qc}};\quad \mathbf{k}_j^c = \mathbf{x}_j\mathbf{W}^{\mathbf{Kc}};\quad \mathbf{v}_j^c = \mathbf{x}_j\mathbf{W}^{\mathbf{Vc}};\quad \forall\, c \quad 1 \le c \le h$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}}$$

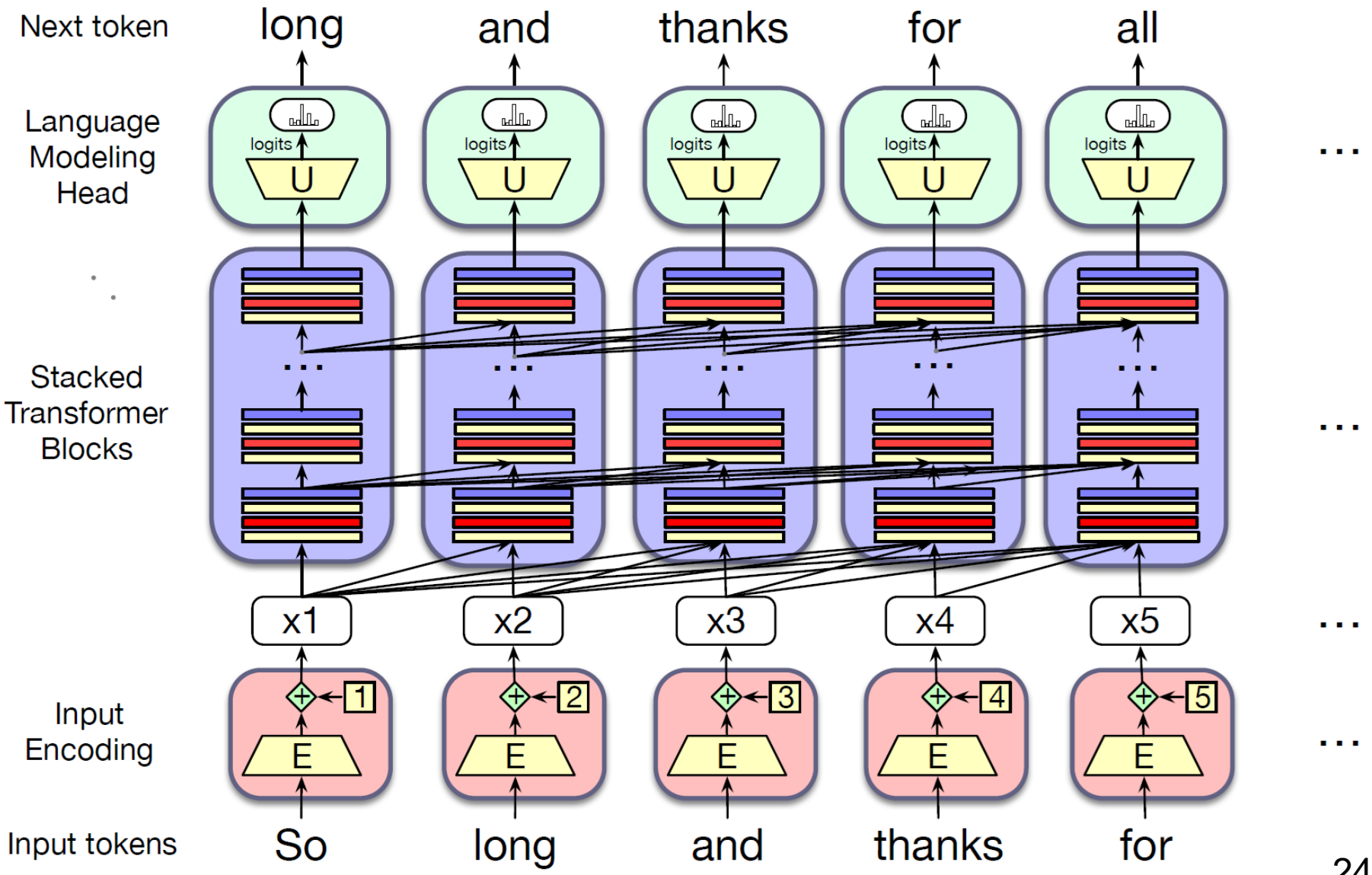
$$\alpha_{ij}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j)) \;\; \forall j \le i$$

$$\mathbf{head}_i^c = \sum_{j \le i} \alpha_{ij}^c \mathbf{v}_j^c$$

$$\mathbf{a}_i = (\mathbf{head}^1 \oplus \mathbf{head}^2 \ldots \oplus \mathbf{head}^h)\mathbf{W}^O$$

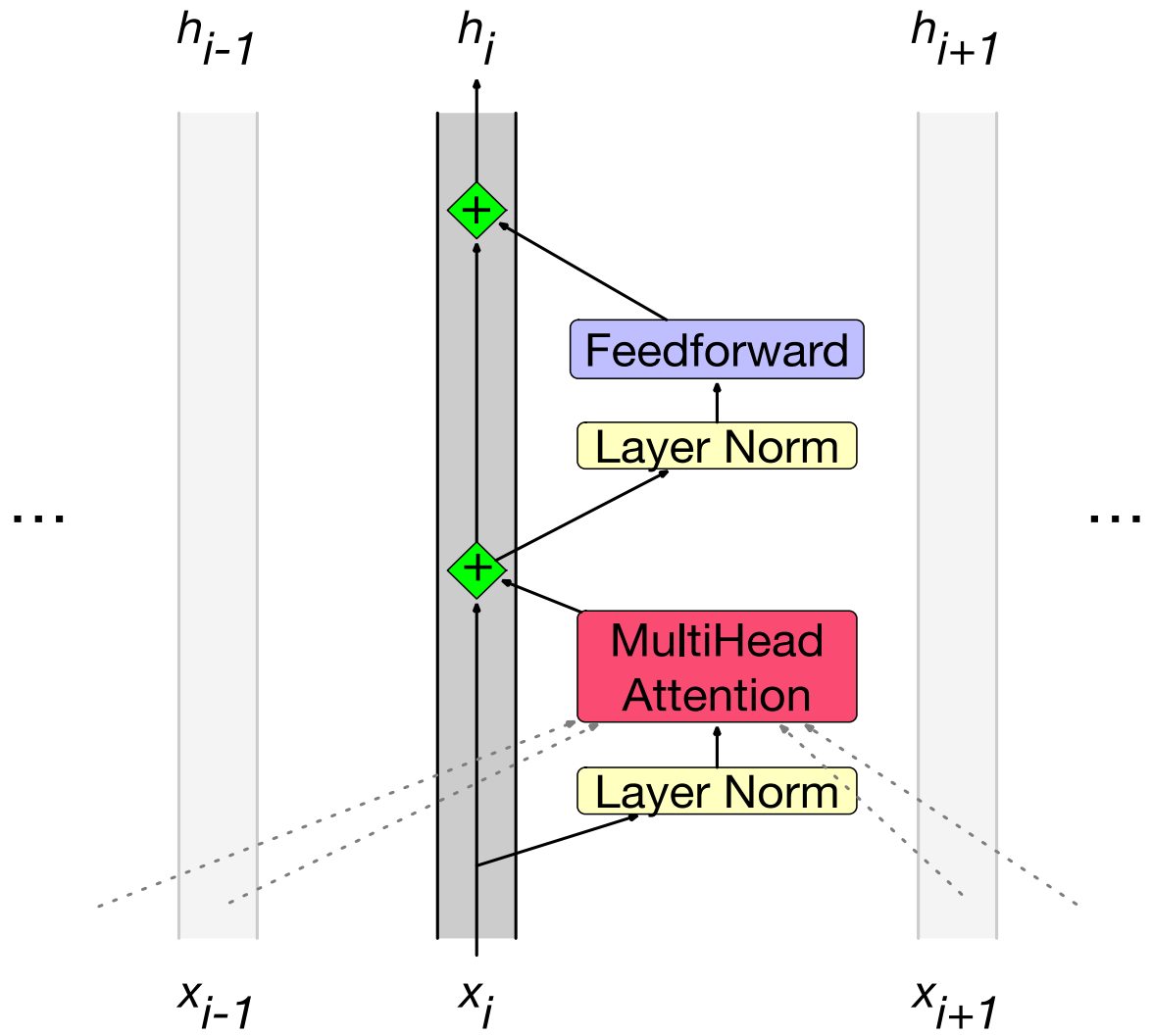
$$\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \cdots, \mathbf{x}_N]) = \mathbf{a}_i$$

Transformer Blocks



Inside a Transformer Block

$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2$$



Layer Norm

Layer norm is a variant of the z-score from statistics, where, bias β and gain γ are weights with the same dimension as $\boldsymbol{x}$.

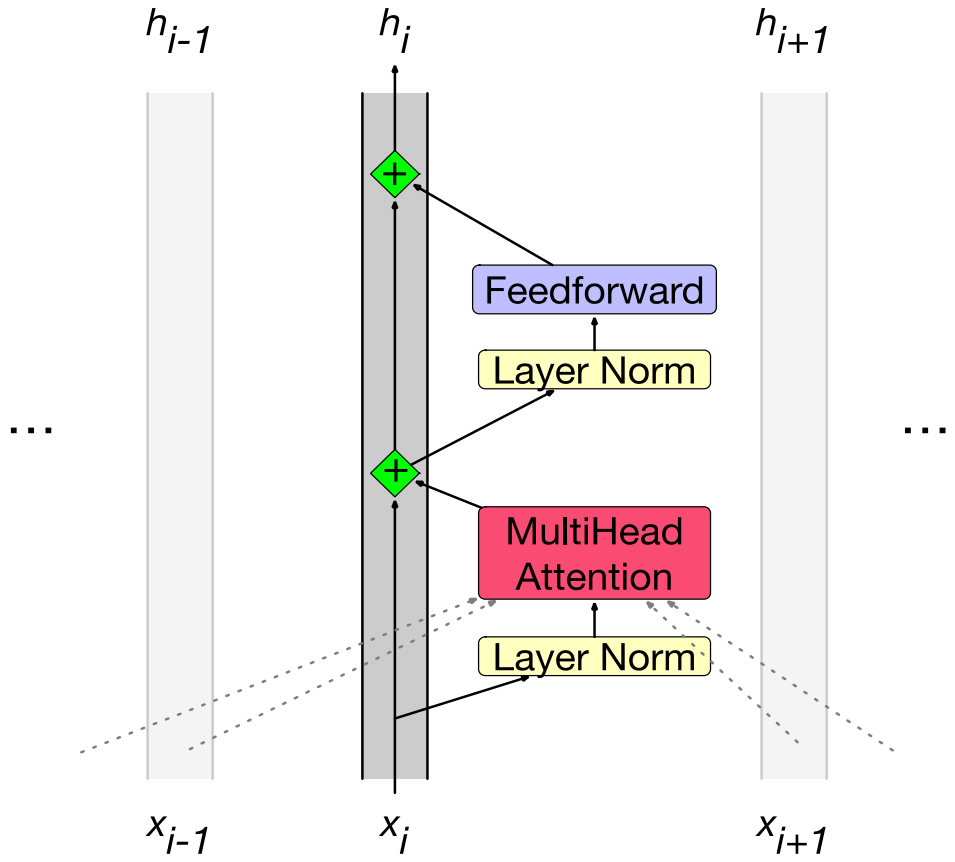
$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$$

$$\sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2}$$

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

$$\text{LayerNorm}(\mathbf{x}) = \gamma\,\frac{(\mathbf{x} - \mu)}{\sigma} + \beta$$

Transformer Block: Summary



$$
\begin{aligned}
\mathbf{t}_i^1 &= \text{LayerNorm}(\mathbf{x}_i) \\
\mathbf{t}_i^2 &= \text{MultiHeadAttention}(\mathbf{t}_i^1, [\mathbf{x}_1^1, \cdots, \mathbf{x}_N^1]) \\
\mathbf{t}_i^3 &= \mathbf{t}_i^2 + \mathbf{x}_i \\
\mathbf{t}_i^4 &= \text{LayerNorm}(\mathbf{t}_i^3) \\
\mathbf{t}_i^5 &= \text{FFN}(\mathbf{t}_i^4) \\
\mathbf{h}_i &= \mathbf{t}_i^5 + \mathbf{t}_i^3
\end{aligned}
$$

Masked Multi-Head Attention

Masked multi-head attention prevents the model from looking at future words when making predictions for the current word. Adding $-\infty$ to cells in upper triangle, softmax will turn them to 0.

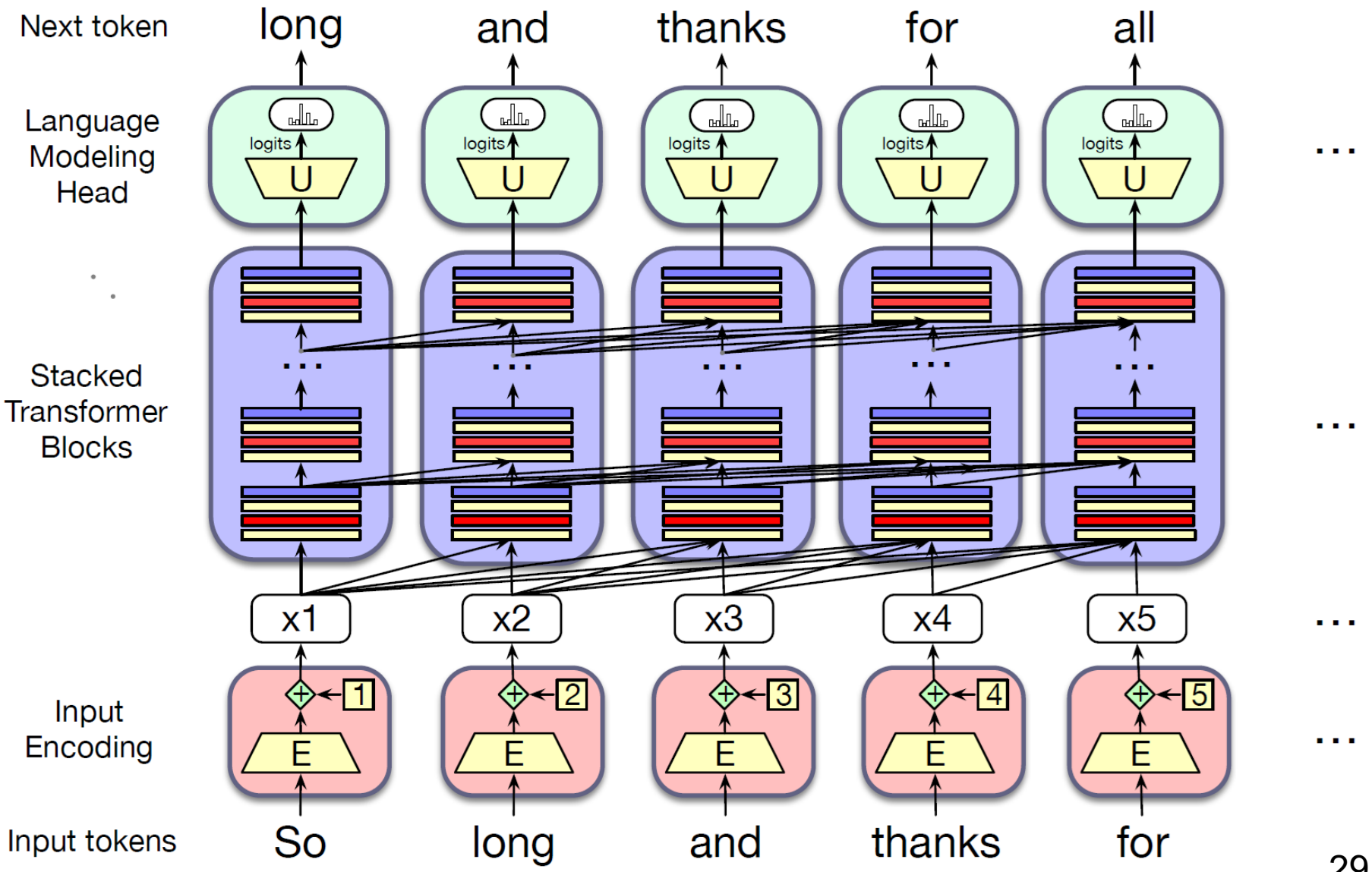
$$\mathbf{A} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right)\right)\mathbf{V}$$

q1·k1	−∞	−∞	−∞
q2·k1	q2·k2	−∞	−∞
q3·k1	q3·k2	q3·k3	−∞
q4·k1	q4·k2	q4·k3	q4·k4

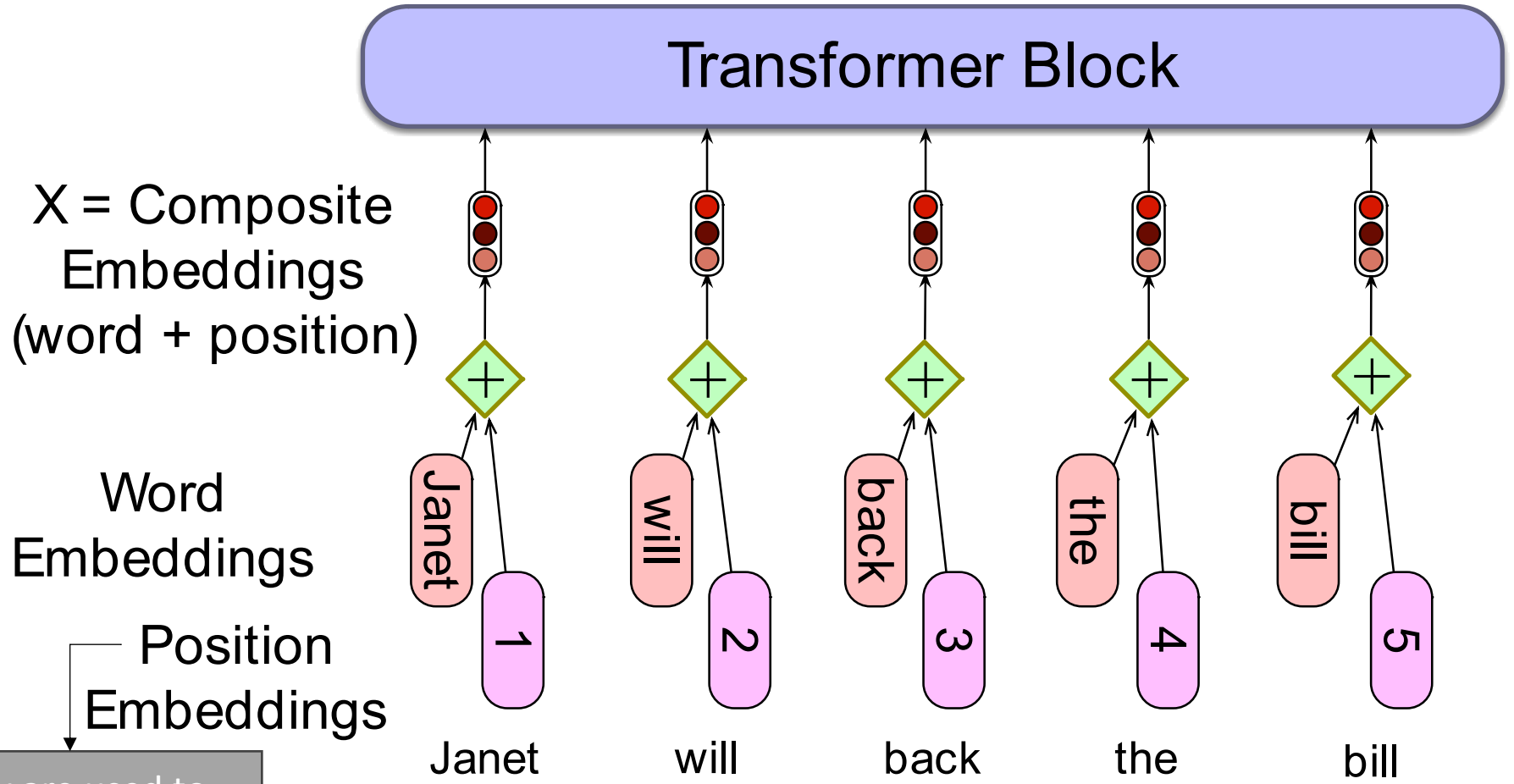
N

N

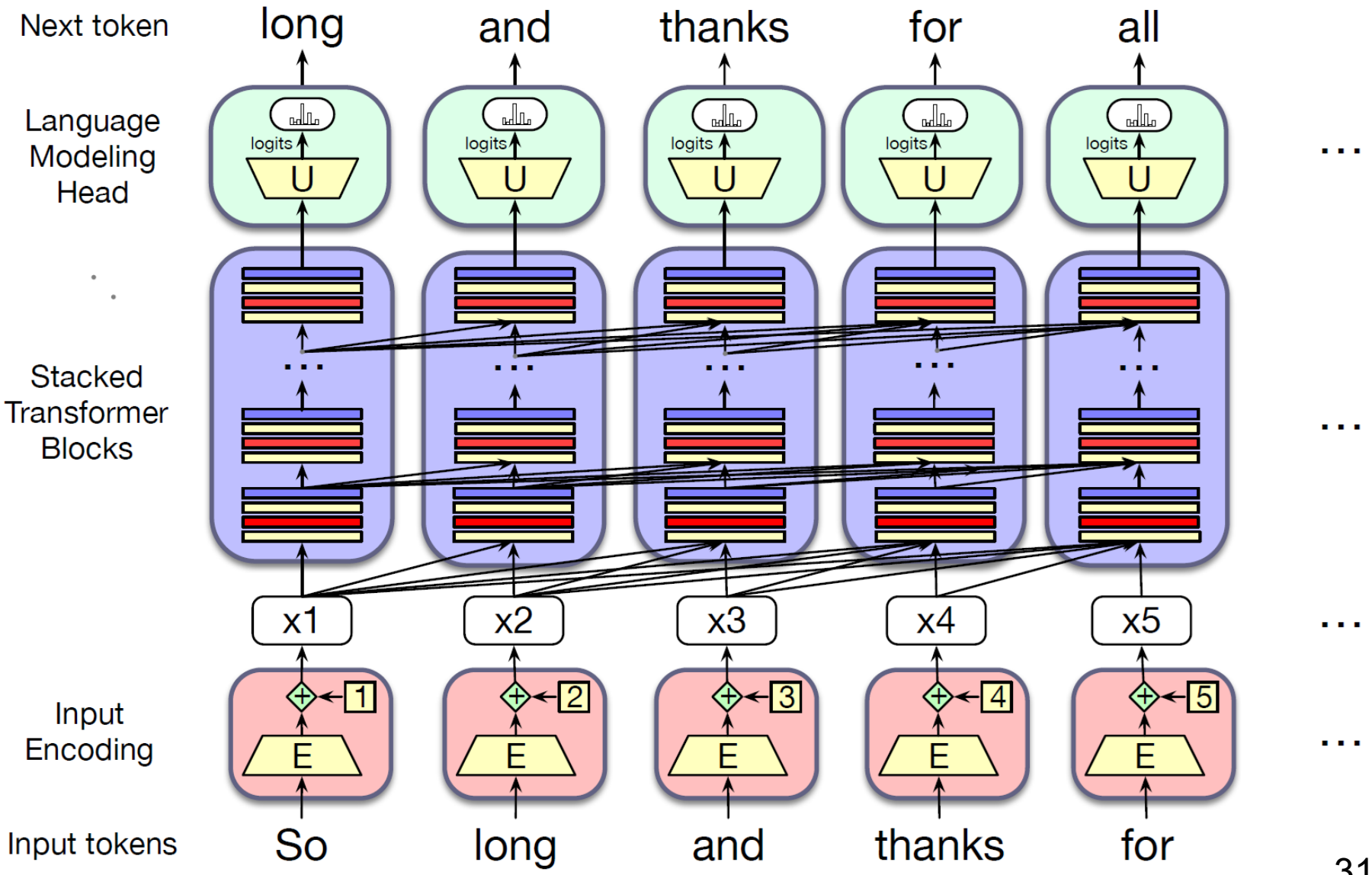
What is Input Encoding?



Input Encoding means Composite Embeddings

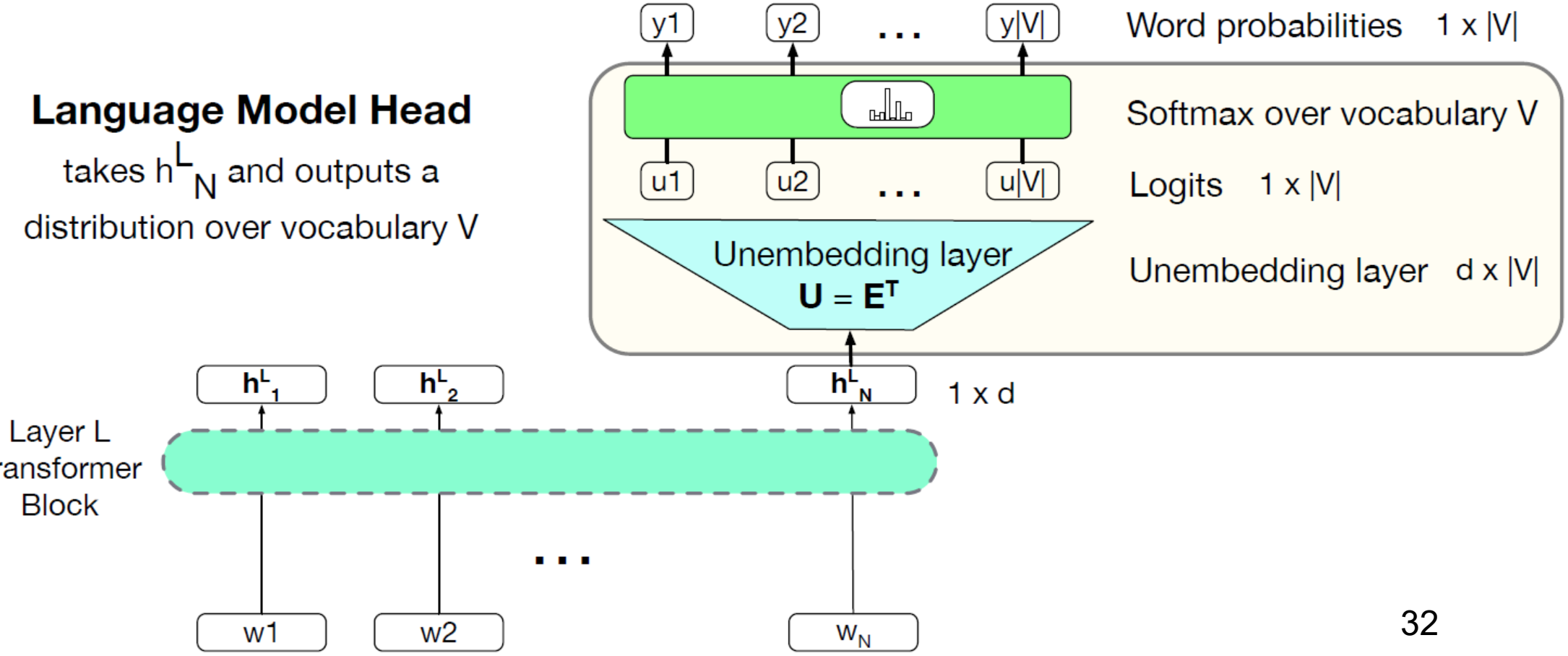


What is Language Modeling Head?



Language Modeling Head

It takes the output of the final/top transformer layer from token N and uses it to predict the next word via $\boldsymbol{y} = softmax(\boldsymbol{u})$, where $\boldsymbol{u} = \boldsymbol{h}_N^L \boldsymbol{E}^T$ maps $\boldsymbol{h}_N^L$ to a probability distribution over vocabulary V.



The Original Transformer

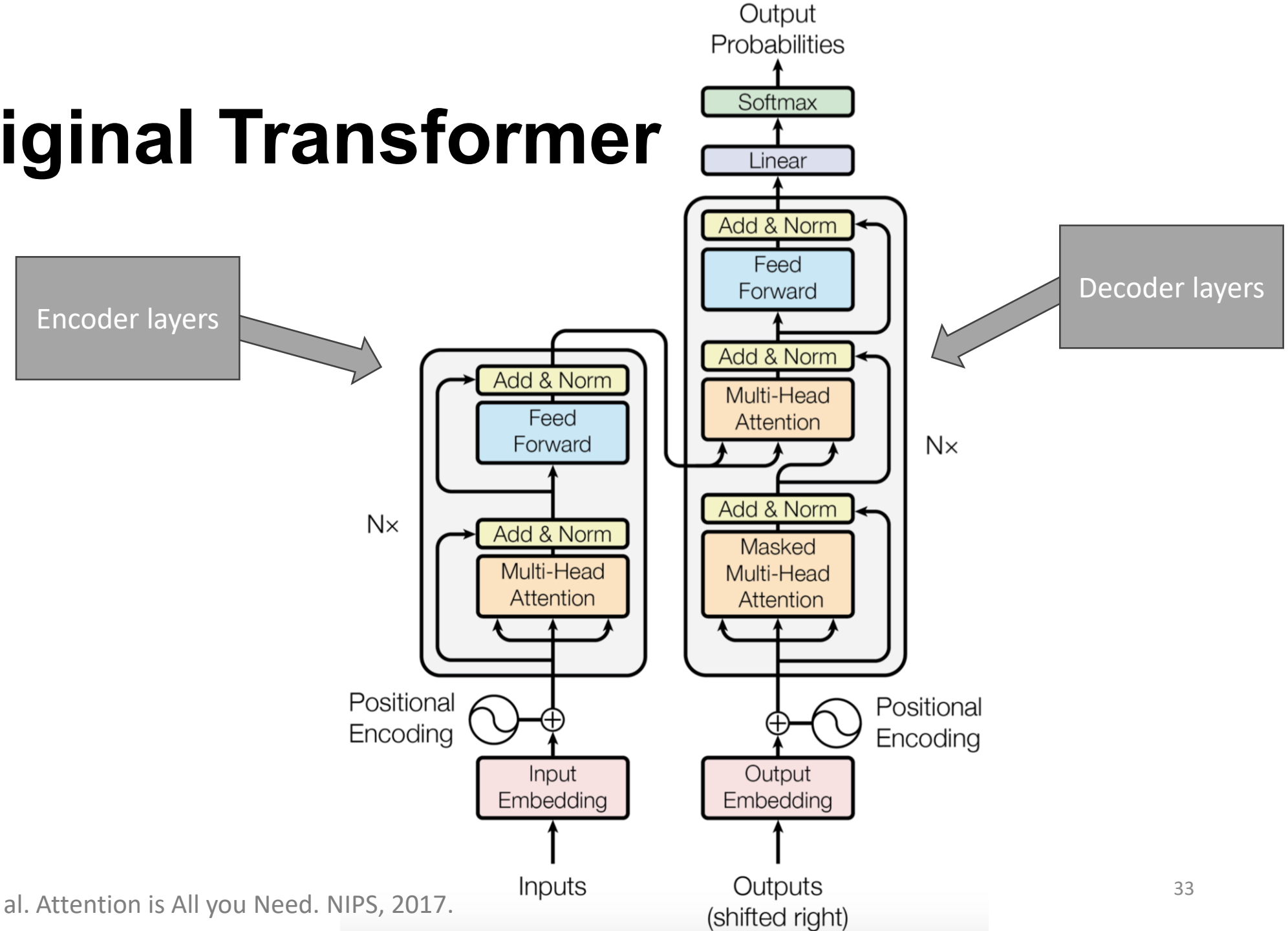
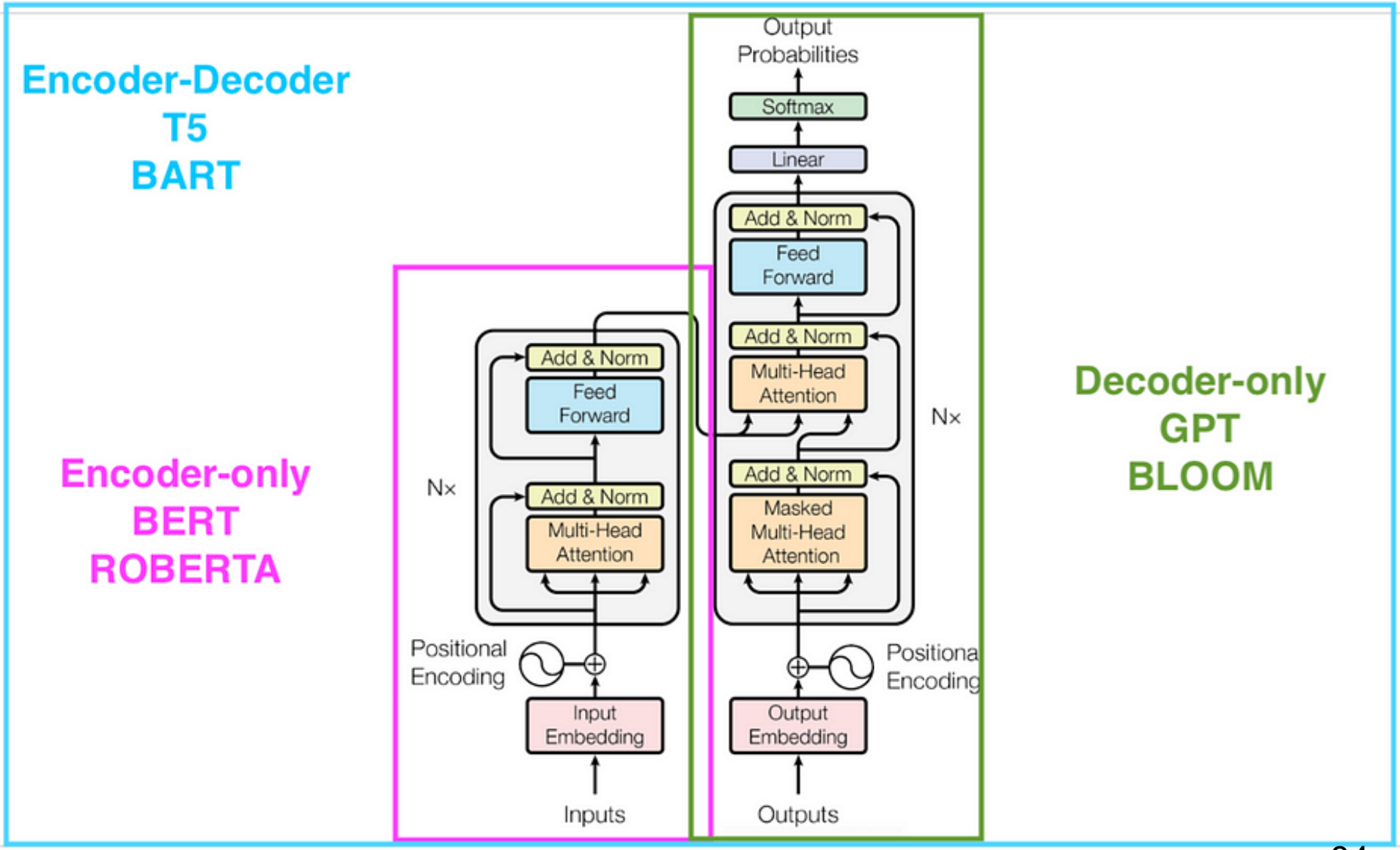


Image fromVaswani, A. et al. Attention is All you Need. NIPS, 2017.

Examples of Transformer Models



Picture from [BERT, GPT and BART: a short comparison | Medium]

What's Next?

- Readings:
  - Jurafsky and Martin. Speech and Language Processing. Chapters 6 and 8, 2026. [recommended]
  - Vaswani, et al. "Attention Is All You Need". NIPS, 2017.

- Workshop **tomorrow**: Neural nets exercise+code.

- Next lecture: Overview of Neural Networks